Building a Root CA and Enabling HTTPS with OpenSSL

Lab 3 Report

Huatai Pan and Rauf Muhammad

NETAUTH – Network Security: Authentication and Authorization

23 October 2026

Abstract

We created an encrypted 4096-bit RSA root certification authority (CA) on Kali Linux, generated a separate 2048-bit RSA key and certificate signing request (CSR) on an Ubuntu Flask server, and issued a three-year TLS server certificate. We then configured the existing web application to serve HTTPS on port 8443. OpenSSL verified the certificate chain, a client request using the root certificate returned HTTP 200, and Firefox accepted the connection after the laboratory root CA was imported. The server private key remained on Ubuntu; the root CA private key remained on Kali.

1 Objective and laboratory environment

The objective was to protect communication with the existing SEC-LAB Flask application using TLS and a certificate issued by a laboratory CA. The application was initially served over unencrypted HTTP on port 5000. We retained that configuration until the certificate had been created and verified, then changed the Flask startup configuration to serve HTTPS on port 8443. Table 1 records the laboratory roles.

Table 1: Laboratory systems and certificate roles.

System	Role	Lab address	Relevant location
Kali Linux	Root CA and browser client	192.168.1.5	`/home/netauth/lab-pki`
Ubuntu Server	Existing Flask web server	192.168.1.35	`/home/netauth/insecure-server`

The CA key and certificate were stored under `ca/` on Kali. The server key and CSR were created under `server/` on Ubuntu. Only the CSR was sent to Kali for signing; only the issued certificate was sent back to Ubuntu. Neither private key was transferred between machines.

2 Part I: Create and inspect the root CA

On Kali, we created a dedicated working directory and generated an AES-256-encrypted 4096-bit RSA private key with `openssl genrsa -aes256`. We restricted its file permissions to mode 600. The private-key passphrase was required again when creating the root certificate and issuing the server certificate; it is not included in this report. No private-key dump is reproduced here.

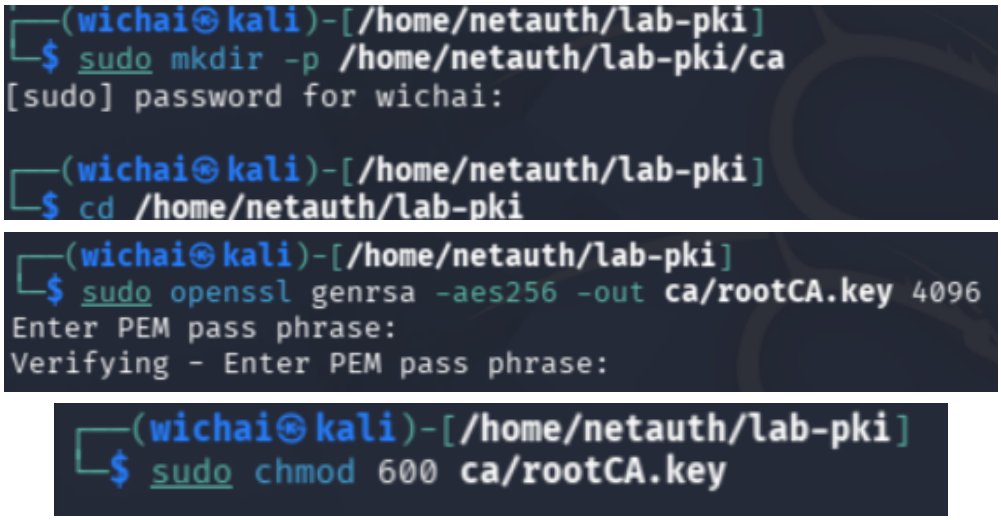


Figure 1: Preparing the Kali CA directory, generating the encrypted root key, and setting mode 600. No passphrase or private-key material is shown.

We created a self-signed root certificate valid for 5475 days (approximately 15 years), with critical `CA:TRUE` basic constraints and `keyCertSign,cRLSign` key usage. The observed subject and issuer were both `C=ES, O=SEC-LAB, CN=SEC-LAB Root CA`; the validity interval was 23 October 2026 to 19 October 2041.

```
┌──(wichai㉿kali)-[/home/netauth/lab-pki]
└─$ openssl req -x509 -new -sha256 -days 5475 \
  -key ca/rootCA.key -out ca/rootCA.crt \
  -subj "/C=ES/O=SEC-LAB/CN=SEC-LAB Root CA" \
  -addext "basicConstraints=critical,CA:TRUE" \
  -addext "keyUsage=critical,keyCertSign,cRLSign" \
  -addext "subjectKeyIdentifier=hash"
Enter pass phrase for ca/rootCA.key:

┌──(wichai㉿kali)-[/home/netauth/lab-pki]
└─$ openssl x509 -in ca/rootCA.crt -noout -subject -issuer -dates
subject=C=ES, O=SEC-LAB, CN=SEC-LAB Root CA
issuer=C=ES, O=SEC-LAB, CN=SEC-LAB Root CA
notBefore=Oct 23 11:25:32 2026 GMT
notAfter=Oct 19 11:25:32 2041 GMT
```

Figure 2: Root CA creation and matching subject and issuer with a 15-year validity period.

```
┌──(wichai㉿kali)-[/home/netauth/lab-pki]
└─$ openssl x509 -in ca/rootCA.crt -text -noout | grep -E -A2 'Public Key Algorithm|Public-Key:|Basic Constraints|Key Usage'
            Public Key Algorithm: rsaEncryption
                Public-Key: (4096 bit)
                Modulus:
--
            X509v3 Basic Constraints: critical
                CA:TRUE
            X509v3 Key Usage: critical
                Certificate Sign, CRL Sign
            X509v3 Subject Key Identifier:
```

Figure 3: Root CA public key and extensions: 4096-bit RSA, `CA:TRUE`, Certificate Sign and CRL Sign.

3 Part II: Generate the server key and CSR

On Ubuntu, we generated a separate 2048-bit RSA private key and set its permissions to mode 600. The CSR configuration specified `C=ES`, `O=NETAUTH`, and `CN=netauth.server`. Its requested subject alternative names (SANs) were `DNS:netauth.server` and `IP:192.168.1.35`. OpenSSL reported `Certificate request self-signature verify OK`, confirming that the CSR was signed by the server's private key. We transferred `server.csr` to Kali via SCP and verified it again there. The server's `server.key` stayed on Ubuntu.

```
netauth@serv1:~$ cd /home/netauth/insecure-server
netauth@serv1:~/insecure-server$ pwd
/home/netauth/insecure-server
netauth@serv1:~/insecure-server$ mkdir -p server
netauth@serv1:~/insecure-server$ openssl genrsa -out server/server.key 2048
netauth@serv1:~/insecure-server$ chmod 600 server/server.key
netauth@serv1:~/insecure-server$ ls -l server/server.key
-rw------- 1 netauth netauth 1704 Oct 23 11:43 server/server.key
netauth@serv1:~/insecure-server$ cat > server/server.cnf <<'EOF'
[req]
prompt = no
distinguished_name = dn
req_extensions = req_ext

[dn]
C = ES
O = NETAUTH
CN = netauth.server

[req_ext]
subjectAltName = @alt_names

[alt_names]
DNS.1 = netauth.server
IP.1 = 192.168.1.35
EOF
netauth@serv1:~/insecure-server$ openssl req -new -sha256 -key server/server.key -out server/server.csr -config server/server.cnf
netauth@serv1:~/insecure-server$ openssl req -in server/server.csr -verify -noout
Certificate request self-signature verify OK
netauth@serv1:~/insecure-server$ openssl req -in server/server.csr -noout -subject -text | grep -A2 'Subject Alternative Name'
                X509v3 Subject Alternative Name:
                    DNS:netauth.server, IP Address:192.168.1.35
    Signature Algorithm: sha256WithRSAEncryption
```

Figure 4: Ubuntu server key, CSR configuration, CSR verification, and requested SANs.

```
scp server/server.csr wichai@192.168.1.5:/home/netauth/lab-pki/server/
The authenticity of host '192.168.1.5 (192.168.1.5)' can't be established.
ED25519 key fingerprint is SHA256:x3HAgDn2vRSw71EgZlGI6/zQ1sm19jeomjqqpvR+XPM.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.1.5' (ED25519) to the list of known hosts.
wichai@192.168.1.5's password:
server.csr                                                                    100% 1009     1.1MB/s   00:00
```

Figure 5: Transfer of the CSR from Ubuntu to the Kali CA over the laboratory network.

```
┌──(wichai㉿kali)-[/home/netauth/lab-pki]
└─$ cd /home/netauth/lab-pki
openssl req -in server/server.csr -verify -noout
Certificate request self-signature verify OK
```

Figure 6: CSR signature verification after its arrival on Kali.

4 Part III: Issue and verify the server certificate

On Kali, the CA extension file set critical `CA:FALSE`, digital-signature and key-encipherment usage, and `serverAuth` extended key usage. The handout's CSR example names `netauth.server`, whereas its final checklist also asks for `sec-lab.local`. We retained the originally requested name and added `DNS:sec-lab.local` to the CA's issuance extensions. Therefore, the final certificate contains both DNS names and `IP:192.168.1.35`. This addition was made during CA issuance; the original CSR was not altered.

Using the root CA key, we issued a server certificate valid for 1095 days (approximately three years), shorter than the root certificate's validity. The final certificate has subject `C=ES, O=NETAUTH, CN=netauth.server`, issuer `SEC-LAB Root CA`, `CA:FALSE`, and TLS Web Server Authentication usage. The verification command and its observed result were:

```
openssl verify -CAfile ca/rootCA.crt server/server.crt
server/server.crt: OK
```

```
┌──(wichai㉿kali)-[/home/netauth/lab-pki]
└─$ cat > server/server.ext <<'EOF'
basicConstraints = critical,CA:FALSE
keyUsage = critical,digitalSignature,keyEncipherment
extendedKeyUsage = serverAuth
subjectKeyIdentifier = hash
authorityKeyIdentifier = keyid,issuer
subjectAltName = @alt_names

[alt_names]
DNS.1 = netauth.server
DNS.2 = sec-lab.local
IP.1 = 192.168.1.35
EOF

┌──(wichai㉿kali)-[/home/netauth/lab-pki]
└─$ openssl x509 -req -sha256 -days 1095 \
  -in server/server.csr \
  -CA ca/rootCA.crt -CAkey ca/rootCA.key \
  -CAcreateserial -out server/server.crt \
  -extfile server/server.ext

openssl verify -CAfile ca/rootCA.crt server/server.crt
openssl x509 -in server/server.crt -noout -ext subjectAltName
Certificate request self-signature ok
subject=C=ES, O=NETAUTH, CN=netauth.server
Enter pass phrase for ca/rootCA.key:
server/server.crt: OK
X509v3 Subject Alternative Name:
    DNS:netauth.server, DNS:sec-lab.local, IP Address:192.168.1.35
```

Figure 7: Final issuance configuration and successful chain verification. The added SAN is visible in the issued certificate.

```
┌──(wichai㉿kali)-[/home/netauth/lab-pki]
└─$ openssl x509 -in server/server.crt -noout -issuer -subject -dates
issuer=C=ES, O=SEC-LAB, CN=SEC-LAB Root CA
subject=C=ES, O=NETAUTH, CN=netauth.server
notBefore=Oct 23 11:58:43 2026 GMT
notAfter=Oct 22 11:58:43 2029 GMT

┌──(wichai㉿kali)-[/home/netauth/lab-pki]
└─$ openssl x509 -in server/server.crt -text -noout | grep -E -A2 'Basic Constraints|Key Usage|Subject Alternative Name'
            X509v3 Basic Constraints: critical
                CA:FALSE
            X509v3 Key Usage: critical
                Digital Signature, Key Encipherment
            X509v3 Extended Key Usage:
                TLS Web Server Authentication
            X509v3 Subject Key Identifier:
--
            X509v3 Subject Alternative Name:
                DNS:netauth.server, DNS:sec-lab.local, IP Address:192.168.1.35
    Signature Algorithm: sha256WithRSAEncryption
```

Figure 8: Fields of the final server certificate: issuer, subject, dates, constraints, server authentication, and SANs.

5 Part IV: Deploy HTTPS and establish browser trust

We copied the issued `server.crt` back to Ubuntu. The public key extracted from that certificate and the public key derived from `server.key` produced identical SHA-256 hashes, confirming that the certificate matched the private key on the server. After backing up `app.py`, we changed its Flask startup block to load `server/server.crt` and `server/server.key` on port 8443 with debug mode disabled. The `sec-lab` service restarted and remained active.

```
app.run(host="0.0.0.0", port=8443,
        ssl_context=("server/server.crt", "server/server.key"),
        debug=False)
```

From Kali, the following request used the laboratory root certificate to validate the server's chain and address. It returned `HTTP/1.1 200 OK`:

```
curl --cacert ca/rootCA.crt -I https://192.168.1.35:8443/
```

```
[netauth@serv1:~/insecure-server$ ls -l server/server.crt server/server.key
 -rw-rw-r-- 1 netauth netauth 1671 Oct 23 12:07 server/server.crt
 -rw------- 1 netauth netauth 1704 Oct 23 11:43 server/server.key
[netauth@serv1:~/insecure-server$ openssl x509 -in server/server.crt -pubkey -noout | openssl sha256
 SHA2-256(stdin)= 1f13a69ab14dbc06b9f8822d72bcd5e7b809749293db5438d5199f1e0d45531c
[netauth@serv1:~/insecure-server$ openssl pkey -in server/server.key -pubout | openssl sha256
 SHA2-256(stdin)= 1f13a69ab14dbc06b9f8822d72bcd5e7b809749293db5438d5199f1e0d45531c
```

Figure 9: Matching SHA-256 hashes of the server certificate and private key public components.

```
[netauth@serv1:~/insecure-server$ sudo systemctl restart sec-lab
[netauth@serv1:~/insecure-server$ systemctl is-active sec-lab
 active
```

Figure 10: The Flask service remained active after the HTTPS configuration was deployed.

```
┌──(wichai㉿kali)-[/home/netauth/lab-pki]
└─$ curl --cacert ca/rootCA.crt -I https://192.168.1.35:8443/
HTTP/1.1 200 OK
Server: Werkzeug/3.1.8 Python/3.12.3
Date: Fri, 23 Oct 2026 12:12:50 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 203
Vary: Cookie
Connection: close
```

Figure 11: HTTPS request succeeds with explicit CA validation and returns HTTP 200.

Firefox initially warned about the untrusted local CA. We imported only `rootCA.crt` into Firefox's Authorities store and enabled trust for identifying websites. We did not add a site-specific security exception. After reloading, Firefox displayed the Flask application at https://192.168.1.35:8443/. Its certificate viewer showed `netauth.server` issued by `SEC-LAB Root CA`; the SAN included the IP address used in the URL.

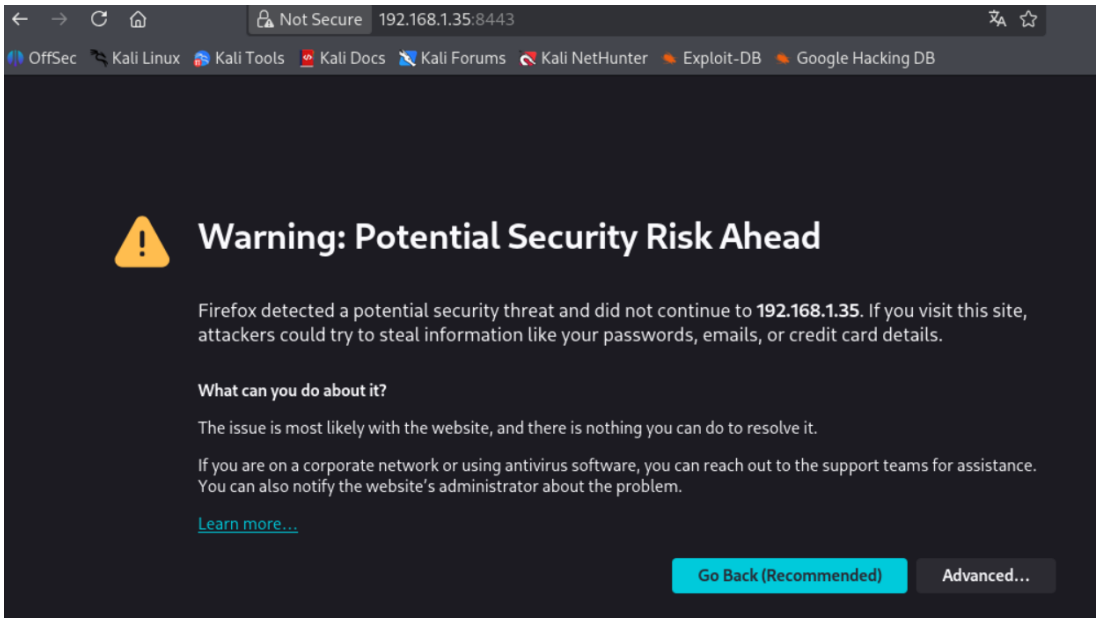


Figure 12: Firefox warning before the laboratory root CA was trusted.

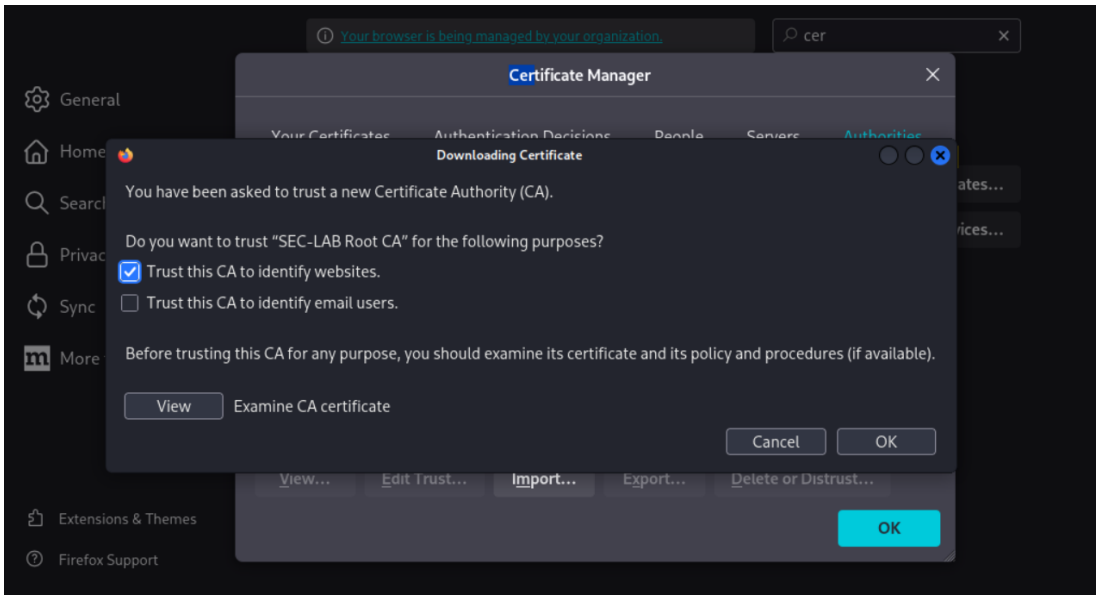


Figure 13: Importing the root CA with trust for website identification only.

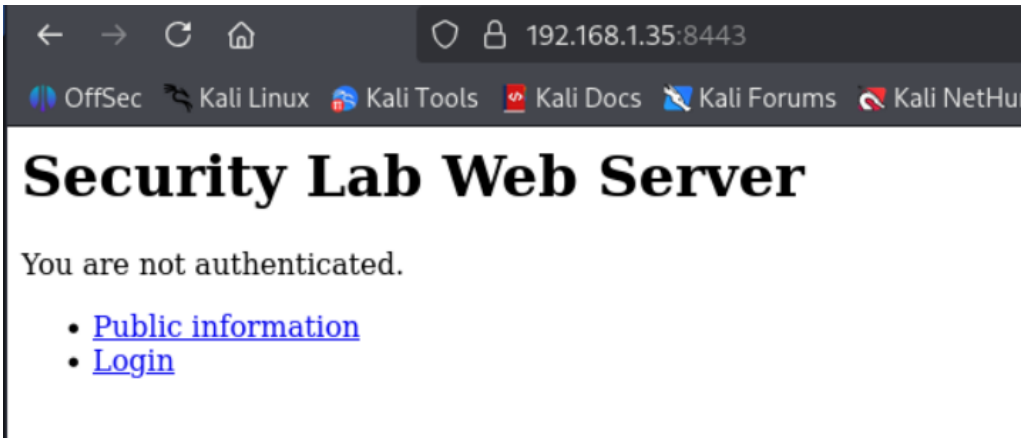


Figure 14: The existing Flask application loads over HTTPS after the root CA is trusted.



Figure 15: Firefox certificate viewer showing the server certificate, root CA, validity, and SANs.

6 Questions and answers

1. **Why protect the root CA private key more carefully than its certificate?** The private key can sign arbitrary certificates that clients may accept as issued by this CA. The root certificate contains the public key and is intentionally distributed to clients. Disclosure of the certificate does not grant signing capability.
2. **Why is the root CA certificate self-signed?** It is the trust anchor and has no superior CA in this laboratory hierarchy. Its own signature confirms internal consistency; actual trust comes from explicitly installing the root certificate on the client.
3. **What is in a CSR, and why does the CA not need the server private key?** The CSR carries the subject, public key, requested attributes such as SANs, and a signature made with the corresponding server private key. The CA verifies the request using the public key and signs the resulting certificate with the CA key. The server private key never needs to leave Ubuntu.
4. **What does the CA signature guarantee?** It binds the signed certificate fields, including the subject and public key, to the issuing CA and allows a client to detect modifications. A client must also trust the CA, check validity and name matching, and rely on the CA's identity checks; the signature alone does not prove that the server software is secure.
5. **Why must the server certificate have `CA:FALSE`?** It identifies the certificate as an end-entity certificate and prevents it from being accepted as a subordinate CA for signing other certificates.
6. **What is the purpose of `serverAuth`?** This extended key usage authorizes the certificate for TLS server authentication, which is the purpose required for the HTTPS endpoint.
7. **Why is SAN important for HTTPS validation?** Clients match the host name or IP used in the URL against SAN entries in the server certificate. The common name alone is not sufficient for modern HTTPS name checking.
8. **What if the client uses an IP absent from SAN?** The client reports a certificate name mismatch and rejects or warns about the connection, even when the issuing root CA is trusted.
9. **Why does installing `rootCA.crt` change Firefox's decision?** It makes the root CA a local trust anchor. Firefox can then validate the server certificate's signature and chain against that root; it still checks the date, intended use, and requested address.
10. **Would HTTPS protect the password database after server compromise?** No. TLS protects data in transit between browser and server. A compromise of the server can expose files, application data, and private keys on the server; separate at-rest protection, access control, and hardening are required.

7 Conclusion and submitted artifacts

The original application was successfully served over HTTPS on port 8443 with a certificate issued by the laboratory root CA. Both OpenSSL and Firefox validated the final chain, and the browser warning disappeared only after the root certificate was trusted. The result protects the client–server channel; it does not repair the application's intentionally insecure authentication or protect server-side data after a host compromise.

The accompanying `artifacts/` directory contains the root certificate, server CSR, server certificate, and both OpenSSL configuration files. No private keys are included.

A CSR and issuance configuration

These are the configuration files used in the experiment. The CSR requests `netauth.server`; the CA extension file also includes `sec-lab.local` for the handout's final checklist.

server.cnf

```
[req]
prompt = no
distinguished_name = dn
req_extensions = req_ext

[dn]
C = ES
O = NETAUTH
CN = netauth.server

[req_ext]
subjectAltName = @alt_names

[alt_names]
DNS.1 = netauth.server
IP.1 = 192.168.1.35
```

server.ext

```
basicConstraints = critical,CA:FALSE
keyUsage = critical,digitalSignature,keyEncipherment
extendedKeyUsage = serverAuth
subjectKeyIdentifier = hash
authorityKeyIdentifier = keyid,issuer
subjectAltName = @alt_names

[alt_names]
DNS.1 = netauth.server
DNS.2 = sec-lab.local
IP.1 = 192.168.1.35
```